nanoCalc-GPT

Lochan Jangid*
(AI/ML INTERN)
Regex Software Services
lochanjangidcoder@gmail.com

Abstract

This is a short introductory paper on my new project, nanoCalc-GPT. The main goal is to understand transformers more deeply by building one and experimenting with it. After building a nanoGPT while following Andrej Karpathy's Zero to Hero series, I wanted to go further and train a transformer on a pattern we all learned long ago: addition. The project also follows an exercise suggested in the description of Karpathy's GPT video. The model reaches 1.000 exact match on held-out addition, and 0.98 overall when extended to all four operations. Please read the paper and share your feedback and suggestions.

---

* Lochan Jangid is a self-taught developer and a B.Sc. Mathematics student at the UOK, currently learning as an AI/ML intern at Regex Software Services. His work focuses on machine learning and data science, with strong skills in Python and SQL. Besides this project, he has built a scalar autograd engine while following Karpathy's Zero to Hero series and published an automated EDA package on PyPI. He is aiming for an ML/AI software engineering role. This is an independent project that has not been peer reviewed, and feedback is welcome.

- Repo: github.com/LochanJangid/nanocalc-gpt
- Linkedin: linkedin.com/in/lochan-jangid

1 Introduction

nanoCalc-GPT is a small transformer trained only on arithmetic examples, predicting the next token. I started with addition of two numbers and then extended the same model to all four operations (+, -, *, /). The central design goal was to stop the model from memorizing and force it to find the pattern. Every decision below comes from a problem I hit while training, and each one is a change to the data or the loss rather than to the model size.

2 Architecture

model architecture is decoder only transformer with 4 transformer blocks. use OneCycleLR for learning rate scheduling and the target is next-token prediction. talk about data architecture they are fromated in fixed width with zero-padded operands and rhs (result) is reversed **(why? see in 3.1)**



(in MATH)

11+06=17



(in training set)

11+06=+710

3 Problem-Decision

3.1 Addition runs right to left, but GPT reads left to right

PROBLEM	On normal expressions like 11+06=17 the answer is computed starting from the least significant digit, but a GPT generates left to right.
WHY IT MATTERS	When the model writes the first answer digit it has not seen the carry that digit depends on. Left to right, there is no stable pattern to learn.
DECISION	Keep LHS as written and reverse the RHS, so the model will read in same direction as generation. now each digit can attend previous result digit with LHS.

3.2 Variable-length examples

PROBLEM	Examples naturally have different lengths (1+2=3 versus 0+15=15), and in a concatenated training stream it is unclear where one example ends and the next begins.
WHY IT MATTERS	If the same position means different things in different examples, the model has to learn alignment before it can learn arithmetic.
DECISION	Use a fixed-width format with zero-padded operands, so every example has the same length and each position keeps the same role (tens digit, ones digit, operator, and so on).

3.3 Loss on the whole expression

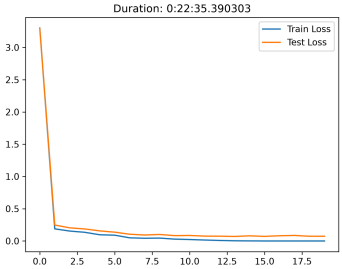
PROBLEM	Computing the loss over every token makes the model also try to predict the operands and the operator
WHY IT MATTERS	Those tokens are random, so there is nothing to learn from them. They add noise to the gradient and pull capacity away from the real task.
DECISION	Mask the loss so that only the result tokens are scored. The only thing left to minimize is the relationship between the left side and the right side, which is how addition works.

4 Evaluation

Embeddings are size of 64 and 4 Heads, 4 transformer blocks. in Train 203k parameters on 36k training examples with 30k steps, batch_size 128, AdamW optimzer and lr_schedular OneCycleLR with weight decay 0.01.

loss on test set: 0.0734

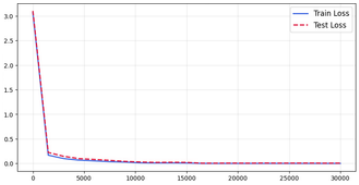
Operator	Accuracy on Test Set
+ (Addition)	100.00 %
- (Subtraction)	99.90%
* (Multiplication)	74.90%
/ (Division)	80.70%



Changes: n_emb 64 → 128, n_block 4 → 6, and the reversal is turned off for the / operator, because a division result is calculated left to right (most significant digit first).

loss on test set: 0.0052

Operator	Accuracy on Test Set
+ (Addition)	100.00 %
- (Subtraction)	100.00 %
* (Multiplication)	99.30%
/ (Division)	85.96%



5 Fun Fact



When we first learned addition, our teacher told us the pattern and the mechanism directly. The poor model gets no explanation. It has to discover both on its own, with only the loss on its wrong answers as feedback.